

Software Design and Analysis Introduction

Instructor: Muhammad Zeeshan Nazar

What is Software Engineering?

- *Software engineering* is the process of **solving customers' problems** by the systematic development and evolution of large, high-quality software systems within cost, time and other constraints.

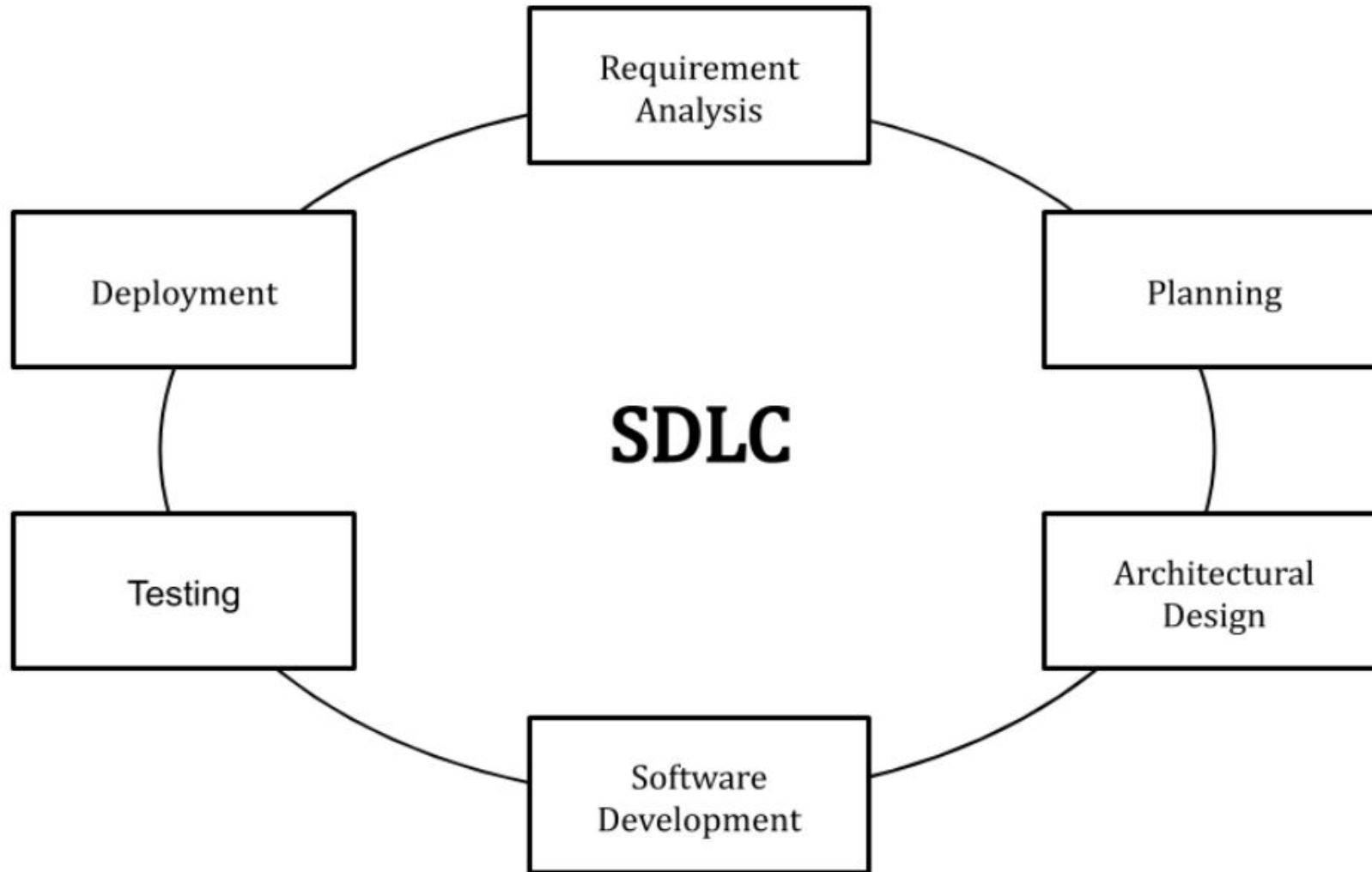
Stakeholders in Software Engineering

- **Users.** These are the people who will use the software. Their goals usually include doing enjoyable or interesting work and gaining recognition for the work they have done.
- **Customers** (also known as *clients*). These are the people who make the decisions about ordering and paying for the software. They may or may not be users – the users may work for them.
- **Software developers.** These are the people who develop and maintain the software, many of whom may be called software engineers.
- **Development managers.** These are the people who run the organization that is developing the software.

Software Development Lifecycle (SDLC)

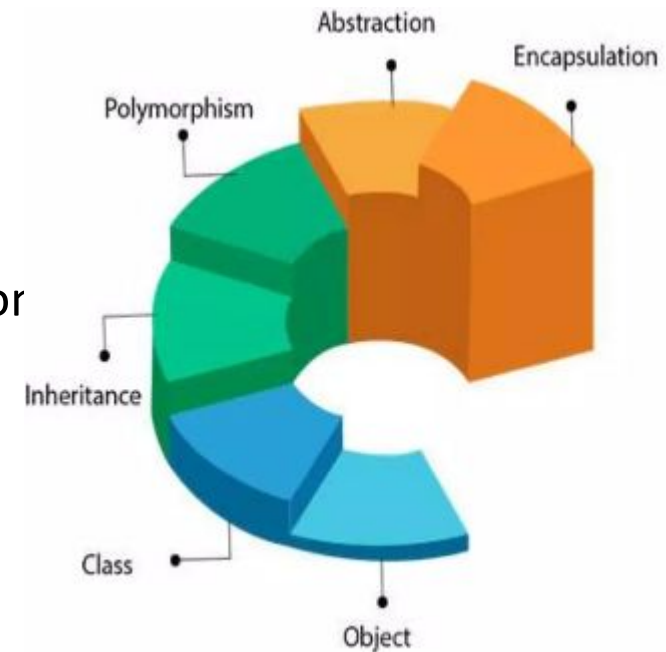
- The **Software Development Life Cycle (SDLC)** refers to a methodology with clearly defined processes for creating high-quality software. SDLC methodology focuses on the following phases of software development:
 - Requirement analysis
 - Planning
 - Software design such as architectural design
 - Software development
 - Testing
 - Deployment/Maintenance

Software Development Lifecycle (SDLC)



OOP?

- **The object-oriented paradigm** is a programming methodology that promotes the efficient design and development of software systems using **reusable components** that can be quickly and safely assembled into larger systems.
- **The main aim of object-oriented programming** is to implement real-world concepts like:
 - **Object** → Real-world entity
 - **Classes** → Templates / Blueprints
 - **Abstraction** → Visibility controls
 - **Encapsulation** → Data hiding
 - **Inheritance** → Backward compatibility, parent–child relation
 - **Polymorphism** → Many forms



Why OOP?

- Worst thing is that Requirements are always change so:
- Break down requirements into objects with responsibilities, not into functional steps
 - Procedural approach ? object oriented approach
- Easier to model real things
- To make software projects more manageable and predictable.
- For more re-use code and prevent reinvention of wheel every time.

Object Oriented Design

- Object-oriented design is a method of design encompassing the process of **object-oriented decomposition** and a notation for depicting both **logical** and **physical** as well as **static** and **dynamic** models of the system under design.

Object Oriented Analysis

- **Object-oriented analysis** is a method of analysis that examines requirements from the perspective of the classes and objects found in the vocabulary of the problem domain.
- **Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity.
 - It is non primitive data types
 - Its members are access modifiers, objects, methods, instance variables and constructors
- **Object:** An object is an instance of a class, any entity that has state and behavior is known as an object.

Constructors and It's Types

A constructor is a **special method** that is automatically called when an object of a class is created.

- **Default Constructor:** Takes **no parameters**, Initializes objects with default values
- **Parameterized Constructor:** Takes **arguments**, Initializes objects with given values
- **Copy Constructor:** A **copy constructor** is needed to create a **new object by copying the data of an existing object**, instead of just copying the reference.
 - **To avoid shallow copy problems :** Shallow copy copies only references, Deep copy (using copy constructor) copies actual data
 - When objects are passed by value in C++, A copy constructor is automatically called Prevents unexpected behavior and data loss.
 - **To duplicate objects safely:** Useful when Returning objects from functions, Storing objects in collections, Creating backups of objects

Destructor

- A destructor is a special member function of a class that is automatically called when an object is destroyed or goes out of scope. Its main purpose is to free resources used by the object.
- Why do we need a destructor?
 - To release memory
 - To close files
 - To free network/database connections
 - To clean up resources before object removal

OOP Concepts: Association

- Association represents a **general relationship** between two classes. It can be one-way or two-way.
- No ownership implied.
- Lifetime of objects is independent.
- **Example:** Teacher and Student. A teacher can teach many students, and a student can have multiple teachers.
- Both Teacher and Student exist independently.

```
int main() {  
    Student s1("Ali");  
    Student s2("Sara");  
  
    Teacher t1("Mr. Khan");  
    t1.addStudent(&s1);  
    t1.addStudent(&s2);  
  
    t1.showStudents();  
}
```

```
class Student {  
public:  
    string name;  
    Student(string n) : name(n) {}  
};  
  
class Teacher {  
public:  
    string name;  
    vector<Student*> students; // Association: Teacher knows Students  
    Teacher(string n) : name(n) {}  
  
    void addStudent(Student* s) {  
        students.push_back(s);  
    }  
  
    void showStudents() {  
        cout << "Teacher " << name << " has students: ";  
        for (auto s : students) {  
            cout << s->name << " ";  
        }  
        cout << endl;  
    }  
};
```



OOP Concepts: Aggregation

- Aggregation is a **weak “has-a” relationship**.
- Whole and part relationship, but the part can exist independently.
- Represented as a **pointer or reference**.
- **Example:** Department and Professor.

A Department has Professors, but a Professor can exist without a Department.

```
int main() {  
    Professor p1("Dr. Ahmed");  
    Professor p2("Dr. Fatima");  
  
    Department d1("Computer Science");  
    d1.addProfessor(&p1);  
    d1.addProfessor(&p2);  
  
    d1.showProfessors();  
}
```

```
class Professor {  
public:  
    string name;  
    Professor(string n) : name(n) {}  
};  
  
class Department {  
public:  
    string name;  
    vector<Professor*> professors; // Aggregation  
  
    Department(string n) : name(n) {}  
  
    void addProfessor(Professor* p) {  
        professors.push_back(p);  
    }  
  
    void showProfessors() {  
        cout << "Department " << name << " has professors: ";  
        for (auto p : professors) {  
            cout << p->name << " ";  
        }  
        cout << endl;  
    }  
};
```

OOP Concepts: Composition

- Composition is a **strong “has-a” relationship**.
- Whole owns the part.
- Lifetime of part depends on whole.
- Usually implemented with **objects (not pointers)** inside the class.
- **Example:** House and Room. A House **owns** Rooms; if House is destroyed, Rooms are destroyed too.

```
int main() {  
    House h("123 Street");  
    h.addRoom("Living Room");  
    h.addRoom("Bedroom");  
    h.addRoom("Kitchen");  
  
    h.showRooms();  
}
```

```
class Room {  
public:  
    string name;  
    Room(string n) : name(n) {}  
};  
  
class House {  
private:  
    vector<Room> rooms; // Composition: House owns Rooms  
public:  
    string address;  
    House(string addr) : address(addr) {}  
  
    void addRoom(string name) {  
        rooms.push_back(Room(name));  
    }  
  
    void showRooms() {  
        cout << "House at " << address << " has rooms: ";  
        for (auto &r : rooms) {  
            cout << r.name << " ";  
        }  
        cout << endl;  
    }  
};
```

Object Model

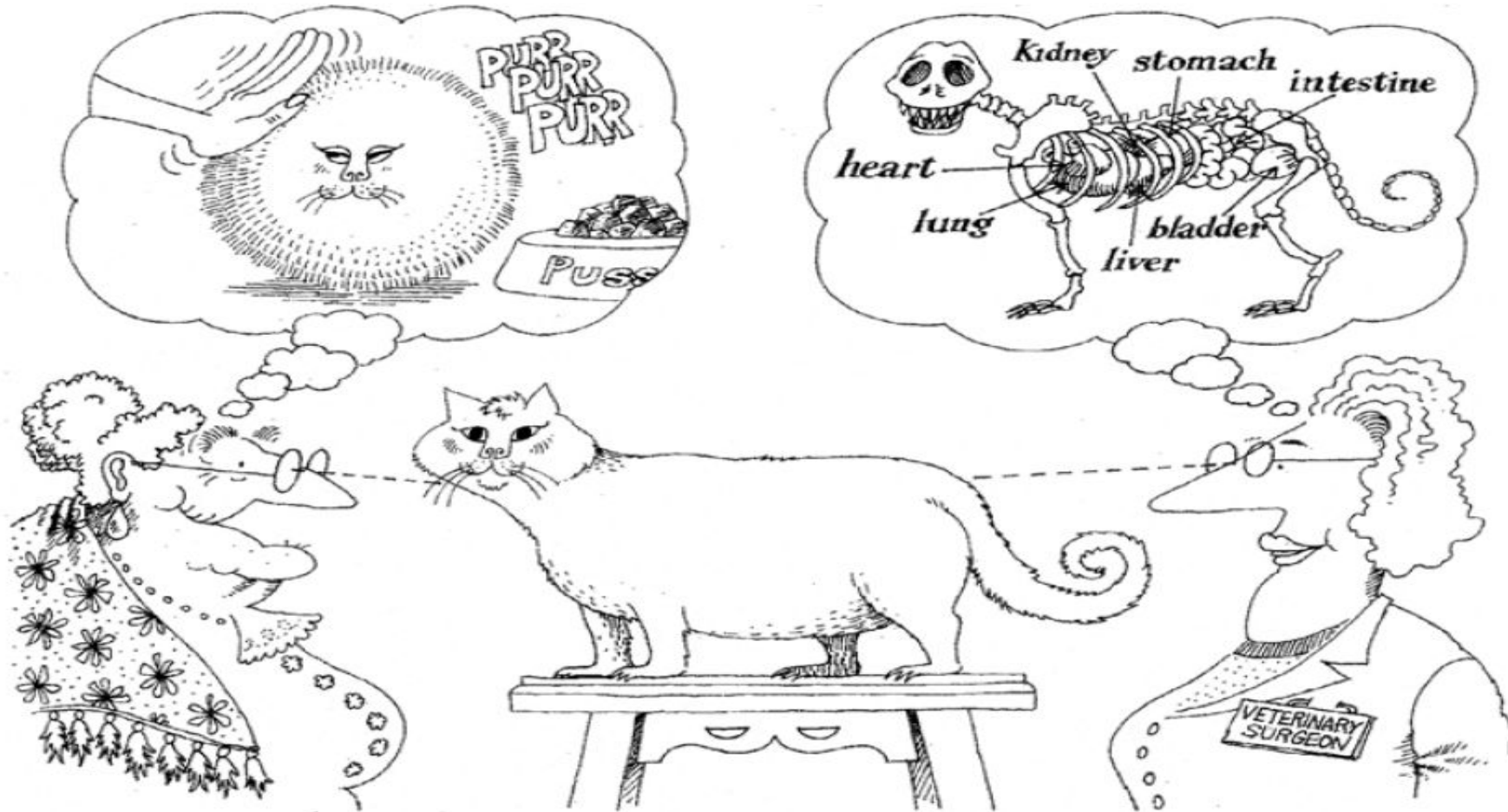
For all things object oriented, the conceptual framework is the object model. There are four major elements of this model:

1. Abstraction
2. Encapsulation
3. Modularity
4. Hierarchy

Abstraction

- An abstraction denotes the essential characteristics of an object that distinguish it from all other kinds of objects and thus provide crisply defined conceptual boundaries, relative to the perspective of the viewer.
- An abstraction focuses on the outside view of an object and so serves to separate an object's essential behavior from its implementation.

Abstraction



Abstraction focuses on the essential characteristics of some object, relative to the perspective of the viewer.

Abstraction

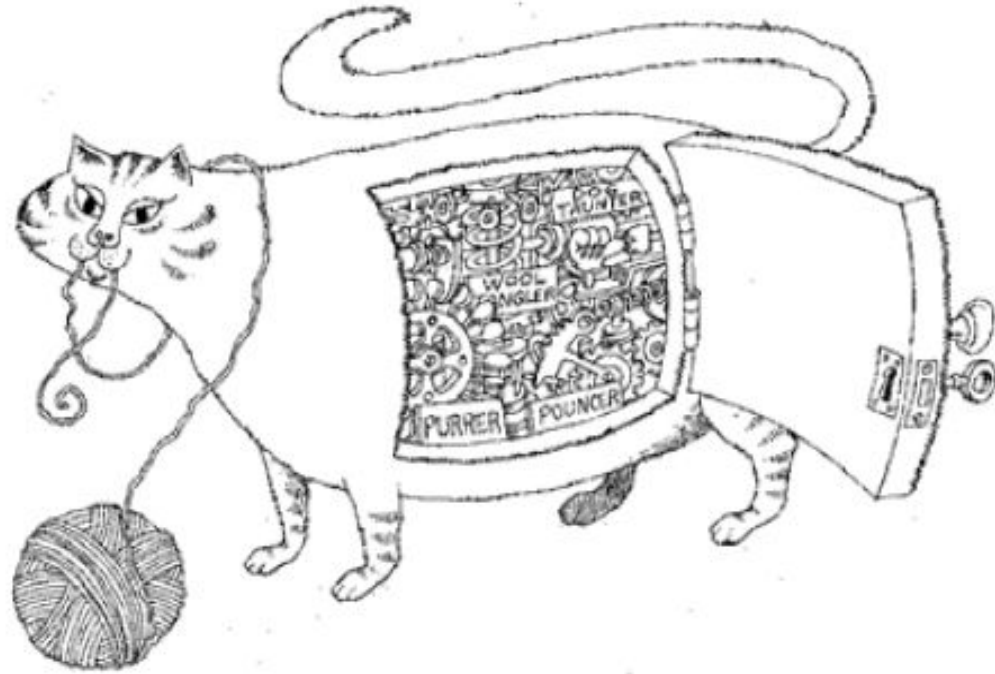
```
class Shape {
public:
    virtual void draw() = 0;    // Pure virtual function
};
// Derived class representing a circle
class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing a circle." << endl;
    }
};
// Derived class representing a rectangle
class Rectangle : public Shape {
public:
    void draw() override {
        cout << "Drawing a rectangle." << endl;
    }
};
```

```
int main() {
    Circle circle;           // Create a Circle object
    Rectangle rectangle;     // Create a Rectangle object
    circle.draw();           // Output: Drawing a circle.
    rectangle.draw();        // Output: Drawing a rectangle.
}
```

Encapsulation

- It refers to the bundling of data (attributes) and methods (functions) that operate on the data into a single unit or class.
- Encapsulation also involves restricting direct access to some of an object's components, which is a way to safeguard the internal state of the object from unintended interference and misuse.

Encapsulation



Encapsulation hides the details of the implementation of an object.

Encapsulation

```
class BankAccount {
private:
    string accountNumber;
    double balance;

public:
    BankAccount(string accNum, double initialBalance) {
        accountNumber = accNum;
        balance = initialBalance;
    }

    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            cout << "$" << amount << " deposited. New balance: $" <<
balance << endl;
        }
    }
}
```

```
void withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
        cout << "$" << amount << " withdrawn. New balance: $" <<
balance << endl;
    } else {
        cout << "Insufficient balance" << endl;
    }
}

double getBalance() {
    return balance;
}

};

int main() {
    BankAccount account("123456789", 500);
    account.deposit(200);
    account.withdraw(100);
    cout << "Account balance: $" << account.getBalance() << endl;
}
```

Modularity

- Partitioning a program creates a number of well-defined, documented boundaries within the program.
- Modularization consists of dividing a program into modules which can be compiled separately, but which have connections with other modules.
- The overall goal of the decomposition into modules is the reduction of software cost by allowing modules to be designed and revised independently

Modularity

```
// BankAccount.h
#ifndef BANKACCOUNT_H
#define BANKACCOUNT_H
#include <string>

class BankAccount {
private:
    string accountNumber;
    double balance;

public:
    BankAccount(string accNum, double initialBalance);
    void deposit(double amount);
    void withdraw(double amount);
    double getBalance() const;
};
#endif // BANKACCOUNT_H
```

Modularity

```
// BankAccount.cpp
```

```
#include "BankAccount.h"
```

```
#include <iostream>
```

```
BankAccount::BankAccount(string accNum,  
double initialBalance) {
```

```
    accountNumber = accNum;
```

```
    balance = initialBalance;
```

```
}
```

```
void BankAccount::deposit(double amount) {
```

```
    if (amount > 0) {
```

```
        balance += amount;
```

```
        cout << "$" << amount << " deposited. New  
balance: $" << balance << endl;
```

```
    }
```

```
}
```

```
void BankAccount::withdraw(double amount) {
```

```
    if (amount > 0 && amount <= balance) {
```

```
        balance -= amount;
```

```
        cout << "$" << amount << " withdrawn. New  
balance: $" << balance << endl;
```

```
    } else {
```

```
        cout << "Insufficient balance" << endl;
```

```
    }
```

```
}
```

```
double BankAccount::getBalance() const {
```

```
    return balance;
```

```
}
```

Modularity

```
// main.cpp
#include <iostream>
#include "BankAccount.h"

int main() {
    BankAccount account("123456789", 500);
    account.deposit(200);
    account.withdraw(100);
    cout << "Account balance: $" << account.getBalance() << endl;
}
```

Hierarchy

- Hierarchy is a ranking or ordering of abstractions.
- The two most important hierarchies in a complex system are its class structure (the “is a” hierarchy) and its object structure (the “part of” hierarchy).
- **Examples of Hierarchy**
 - Single Inheritance (is a hierarchy)
 - Multiple Inheritance (is a hierarchy)
 - ❖ Clashes will occur when two or more super classes provide a field or operation with the same name or signature as a peer superclass.
 - ❖ Repeated inheritance occurs when two or more peer super classes share a common superclass. In such a situation, the inheritance lattice will be **diamond-shaped**, so the question arises, does the leaf class (i.e., subclass) have one copy or multiple copies of the structure of the shared superclass?
 - Aggregation (part of hierarchy)

Hierarchy (Single Inheritance)

```
// Base class
class Animal {
public:
    void eat() {
        cout << "This animal is eating." << endl;
    }
};

// Derived class
class Dog : public Animal {
public:
    void bark() {
        cout << "The dog is barking." << endl;
    }
};

int main() {
    Dog myDog;
    myDog.eat();
    myDog.bark();
}
```

Hierarchy (Aggregation)

```
// Part class
class Engine {
public:
    void start() {
        cout << "Engine started." << endl;
    }
};

// Whole class
class Car {
private:
    Engine engine; // Engine is a part of Car

public:
    void start() {
        engine.start(); // Start the engine when the car starts
        cout << "Car started." << endl;
    }
};
```

```
int main() {
    Car myCar;

    // Car is composed of an Engine, so starting the car also
    starts the engine
    myCar.start(); // Output: Engine started. Car started.

    return 0;
}
```

Diamond Problem

```
class A {  
public:  
    void display() {  
        cout << "A's display()" << endl;  
    }  
};  
class B : public A {  
    // No override of display()  
};  
class C : public A {  
    // No override of display()  
};
```

```
class D : public B, public C {  
    // No override of display()  
};  
int main() {  
    D obj;  
    obj.display();  
    // Which display() should be called?  
    return 0;  
}
```

Solution in C++ (Virtual Inheritance)

```
class A {  
public:  
    virtual void display() {  
        cout << "A's display()" << endl;  
    }  
};  
class B : public virtual A {  
    // No override of display()  
};  
class C : public virtual A {  
    // No override of display()  
};
```

```
class D : public B, public C {  
    // No override of display()  
};  
int main() {  
    D obj;  
    obj.display();  
    // Which display() should be called?  
    return 0;  
}
```

Benefits of Object Model

- Use of the object model:
 - Helps us to exploit the expressive **power of object-based and object-oriented programming languages**
 - Encourages the **reuse** not only of software but of entire designs, leading to the creation of reusable application frameworks
 - Produces systems that are built on stable intermediate forms, which are more **resilient to change**
 - Appeals to the workings of **human cognition**

References

- Object-Oriented Analysis and Design with Applications, Grady Booch et al., 3rd Edition, Pearson, 2007.
- Timothy C. Lethbridge, Robert Laganaiere, Object-Oriented Software Engineering (2nd Edition), McGraw Hill, 2005
- Object-Oriented Modeling and Design with UML, Michael R. Blaha and James R. Rumbaugh, 2nd Edition, Pearson, 2005.